Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи №6

“ Автоматизоване тестування Python-програм ”

Виконала:

Студентка групи ФЕП-33с

Най Анастасія

Перевірив:

Проф. Свелеба С.А.

Львів-2026

Мета роботи: сформувати практичні навички професійного автоматизованого тестування Python-програм із використанням фреймворку pytest, навчитися проєктувати unit та integration tests, створювати fixtures, використовувати параметризацію, перевіряти exceptions, ізолювати файлову систему та зовнішні залежності, застосовувати mocks і аналізувати test coverage.

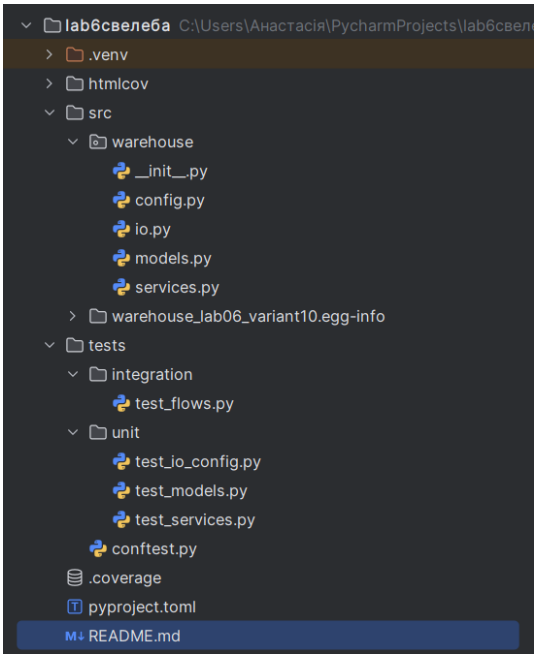
Варіант 10. Система обліку складських запасів

Протестувати:

- створення Product/WarehouseItem та валідацію quantity/price;
- операцію надходження;
- операцію списання;
- insufficient stock;
- пошук за кодом товару;
- розрахунок загальної вартості складу;
- визначення low-stock positions;
- boundary та invalid quantity/unit_price;
- CSV streaming import;
- malformed quantity record;
- JSON export;
- configuration low-stock threshold;
- WarehouseRepository через Mock/fake;
- supplier/reorder service через Mock із side_effect;
- async reorder provider через AsyncMock та integration stock flow.

Хід роботи:

1. Спочатку я створила робочу папку проєкту. Організувала професійну структуру за стандартом src-layout: у папці src/warehouse/ зосередила весь програмний код, залишивши корінь проєкту для конфігураційних файлів (pyproject.toml, README.md) та каталогу тестів tests/ із розділенням на unit/ та integration/.



1. **У файлі pyproject.toml** налаштовано метадані пакета, необов'язкові розробницькі залежності (pytest, pytest-cov, pytest-asyncio), параметри запуску pytest та правила перевірки покриття коду з мінімальним порогом спрацювання fail_under = 80.

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"
[project]
name = "warehouse-lab06-variant10"
version = "0.1.0"
requires-python = ">=3.10"
[project.optional-dependencies]
dev = ["pytest>=8", "pytest-cov>=5"]
[tool.setuptools.packages.find]
where = ["src"]
[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = ["-ra", "--strict-markers"]
markers = ["integration: integration tests"]
[tool.coverage.run]
branch = true
source = ["warehouse"]
[tool.coverage.report]
show_missing = true
fail_under = 80
```

2. **У файлі README.md** оформлено документацію до проєкту та інструкцію для запуску тестів у терміналі.

````
# Lab 06 — Variant 10: Warehouse stock management

Self-contained example for automated testing. Not the original Lab_01 code.

## Run

```bash
python -m pip install -e ".[dev]"
python -m pytest -v --cov=warehouse --cov-branch --cov-report=term-missing --
cov-report=html
```
````

```
Open `htmlcov/index.html` for the coverage report.
```

3. **У файлі src/warehouse/models.py** створено доменні класи WarehouseItem із валідацією полів через метод __post_init__ (перевірка на порожній код/назву, заборона від'ємних значень кількості та ціни) й обчислювальною властивістю total_price, а також клас Warehouse для керування запасами на складі (надходження, списання, перевірка дефіциту).

```
from dataclasses import dataclass

class WarehouseError(Exception):
    """Base warehouse error."""

class InsufficientStockError(WarehouseError):
    pass

class ImportRecordError(WarehouseError):
    def __init__(self, row: int, field: str):
        self.row, self.field = row, field
        super().__init__(f"Invalid {field} at row {row}")

@dataclass
class WarehouseItem:
    code: str
    name: str
    quantity: int
    unit_price: float

    def __post_init__(self):
        if not isinstance(self.code, str) or not self.code.strip():
            raise ValueError("code is required")
        if not isinstance(self.name, str) or not self.name.strip():
            raise ValueError("name is required")
        if type(self.quantity) is not int or self.quantity < 0:
            raise ValueError("quantity must be a nonnegative integer")
        if isinstance(self.unit_price, bool) or not isinstance(self.unit_price,
(int, float)) or not 0 <= self.unit_price < float('inf'):
            raise ValueError("unit_price must be finite and nonnegative")

    @property
    def value(self) -> float:
        return self.quantity * self.unit_price
```

4. **У файлі src/warehouse/config.py** реалізовано зчитування налаштувань середовища (поріг низьких запасів LOW_STOCK_THRESHOLD) за допомогою бібліотеки os та датакласів.

```
import os

def low_stock_threshold():
    raw = os.getenv('WAREHOUSE_LOW_STOCK', '5')
    try:
        threshold = int(raw)
    except ValueError as exc:
        raise ValueError('invalid threshold') from exc
    if threshold < 0:
```

```
        raise ValueError('invalid threshold')
    return threshold
```

5. **У файлі src/warehouse/services.py** зосереджено бізнес-логіку системи (пошук за кодом, розрахунок загальної вартості інвентарю, фільтрація товарів із низьким залишком) та асинхронний сервіс AsyncReorderService для взаємодії із зовнішнім постачальником.

```
from .models import WarehouseItem, InsufficientStockError

class Warehouse:
    def __init__(self, repository=None):
        self.items = {}
        self.repository = repository

    def add(self, item: WarehouseItem):
        if item.code in self.items:
            raise ValueError("duplicate code")
        self.items[item.code] = item
        if self.repository is not None:
            self.repository.save(item)

    def find(self, code: str):
        return self.items.get(code)

    def receive(self, code: str, amount: int):
        self._validate_amount(amount)
        item = self._required(code)
        item.quantity += amount
        return item.quantity

    def issue(self, code: str, amount: int):
        self._validate_amount(amount)
        item = self._required(code)
        if item.quantity < amount:
            raise InsufficientStockError("insufficient stock")
        item.quantity -= amount
        return item.quantity

    @staticmethod
    def _validate_amount(amount):
        if type(amount) is not int or amount <= 0:
            raise ValueError("amount must be positive integer")

    def _required(self, code):
        item = self.find(code)
        if item is None:
            raise KeyError(code)
        return item

    def total_value(self):
        return sum(item.value for item in self.items.values())

    def low_stock(self, threshold: int):
        if type(threshold) is not int or threshold < 0:
            raise ValueError("invalid threshold")
        return [item for item in self.items.values() if item.quantity <=
threshold]

    def request_reorder(self, code, supplier):
        item = self._required(code)
        return supplier.reorder(item.code, item.quantity)
```

```
    async def async_reorder(self, code, provider):
        item = self._required(code)
        return await provider.reorder(item.code, item.quantity)
```

6. **У файлі src/warehouse/io.py** реалізовано експорт та імпорт даних у формати CSV та JSON із перевіркою на пошкоджені записи (*malformed records*).

```
import csv
import json
from pathlib import Path
from .models import WarehouseItem, ImportRecordError

def read_items(path):
    """Stream rows without loading the whole CSV file."""
    with Path(path).open('r', encoding='utf-8-sig', newline='') as file:
        reader = csv.DictReader(file)
        if not reader.fieldnames or not {'code','name','quantity','unit_price'}
<= set(reader.fieldnames):
            raise ImportRecordError(1, 'header')
        for row_number, row in enumerate(reader, 2):
            try:
                quantity = int(row['quantity'])
            except (TypeError, ValueError) as exc:
                raise ImportRecordError(row_number, 'quantity') from exc
            try:
                price = float(row['unit_price'])
            except (TypeError, ValueError) as exc:
                raise ImportRecordError(row_number, 'unit_price') from exc
            try:
                yield WarehouseItem(row['code'], row['name'], quantity, price)
            except (ValueError, TypeError) as exc:
                raise ImportRecordError(row_number, 'item') from exc

def export_json(items, path):
    payload = [{'code': x.code, 'name': x.name, 'quantity': x.quantity,
'unit_price': x.unit_price, 'value': x.value} for x in items]
    Path(path).write_text(json.dumps(payload, ensure_ascii=False, indent=2),
encoding='utf-8')
```

7. **У файлі tests/conftest.py** створено спільні фікстури pytest (sample_item, sample_item_list, sample_warehouse) для ізольованого тестування.

```
import pytest
from warehouse.models import WarehouseItem
from warehouse.services import Warehouse

@pytest.fixture
def item():
    return WarehouseItem('A1', 'Keyboard', 10, 125.5)

@pytest.fixture
def warehouse(item):
    result = Warehouse()
    result.add(item)
    return result

@pytest.fixture
def item_factory():
    def create(code='X', quantity=5, price=10):
        return WarehouseItem(code, 'Sample', quantity, price)
    return create
```

```
@pytest.fixture
def csv_factory(tmp_path):
    def create(content):
        path = tmp_path / 'items.csv'
        path.write_text(content, encoding='utf-8')
        return path
    yield create
```

8. **У файлах tests/unit/ (test_models.py, test_services.py, test_io_config.py)** реалізовано модульні тести для перевірки моделей, валідаторів із параметризацією (@pytest.mark.parametrize), обробки винятків через pytest.raises, числових порівнянь через pytest.approx, а також перевірки взаємодії за допомогою Mock (return_value, side_effect) та AsyncMock.

test_models.py

```
import pytest
from warehouse.models import WarehouseItem

def test_creation(item):
    assert (item.code, item.quantity) == ('A1', 10)

def test_value(item):
    assert item.value == pytest.approx(1255)

@pytest.mark.parametrize('quantity', [0, 1, 100])
def test_valid_quantity(quantity):
    assert WarehouseItem('X', 'X', quantity, 1).quantity == quantity

@pytest.mark.parametrize('quantity', [-1, 1.2, True, '2'])
def test_invalid_quantity(quantity):
    with pytest.raises(ValueError, match='quantity'):
        WarehouseItem('X', 'X', quantity, 1)

@pytest.mark.parametrize('price', [0, 0.01, 100.25])
def test_valid_price(price):
    assert WarehouseItem('X', 'X', 0, price).unit_price == pytest.approx(price)

@pytest.mark.parametrize('price', [-1, float('inf'), float('nan'), True, '10'])
def test_invalid_price(price):
    with pytest.raises(ValueError, match='unit_price'):
        WarehouseItem('X', 'X', 0, price)

@pytest.mark.parametrize('code,name', [('', 'ok'), ('A', ''), (None, 'ok')])
def test_required_fields(code, name):
    with pytest.raises(ValueError):
        WarehouseItem(code, name, 1, 2)
```

test_services.py

```
from unittest.mock import Mock, AsyncMock
import asyncio
import pytest
from warehouse.services import Warehouse
from warehouse.models import InsufficientStockError

def test_empty_warehouse():
    assert Warehouse().total_value() == 0
```

```
def test_find(warehouse, item):
    assert warehouse.find('A1') is item

def test_find_missing(warehouse):
    assert warehouse.find('missing') is None

def test_receive(warehouse):
    assert warehouse.receive('A1', 3) == 13

def test_issue(warehouse):
    assert warehouse.issue('A1', 10) == 0

def test_insufficient_stock(warehouse):
    with pytest.raises(InsufficientStockError, match='insufficient'):
        warehouse.issue('A1', 11)
    assert warehouse.find('A1').quantity == 10

@pytest.mark.parametrize('amount', [0, -1, 2.5, True])
def test_invalid_amount(warehouse, amount):
    with pytest.raises(ValueError):
        warehouse.receive('A1', amount)

def test_missing_code(warehouse):
    with pytest.raises(KeyError):
        warehouse.issue('absent', 1)

def test_low_stock(warehouse):
    assert warehouse.low_stock(9) == []
    assert [x.code for x in warehouse.low_stock(10)] == ['A1']

def test_invalid_threshold(warehouse):
    with pytest.raises(ValueError):
        warehouse.low_stock(-1)

def test_duplicate(warehouse, item_factory):
    with pytest.raises(ValueError, match='duplicate'):
        warehouse.add(item_factory('A1'))

def test_repository_interaction(item):
    repo = Mock()
    warehouse = Warehouse(repo)
    warehouse.add(item)
    repo.save.assert_called_once_with(item)

def test_supplier_return_value(warehouse):
    supplier = Mock()
    supplier.reorder.return value = 'REQ-1'
    assert warehouse.request_reorder('A1', supplier) == 'REQ-1'
    supplier.reorder.assert_called_once_with('A1', 10)

def test_supplier_side_effect(warehouse):
    supplier = Mock()
    supplier.reorder.side_effect = ConnectionError('offline')
    with pytest.raises(ConnectionError, match='offline'):
        warehouse.request_reorder('A1', supplier)

def test_async_reorder(warehouse):
    provider = Mock()
    provider.reorder = AsyncMock(return_value='ASYNC-1')
    assert asyncio.run(warehouse.async_reorder('A1', provider)) == 'ASYNC-1'
    provider.reorder.assert_awaited_once_with('A1', 10)
```

test_io_config.py

```
import json
import pytest
from warehouse.io import read_items, export_json
from warehouse.config import low_stock_threshold
from warehouse.models import ImportRecordError

def test_config_default(monkeypatch):
    monkeypatch.delenv('WAREHOUSE_LOW_STOCK', raising=False)
    assert low_stock_threshold() == 5

def test_config_override(monkeypatch):
    monkeypatch.setenv('WAREHOUSE_LOW_STOCK', '2')
    assert low_stock_threshold() == 2

@pytest.mark.parametrize('raw', ['abc', '-1'])
def test_config_invalid(monkeypatch, raw):
    monkeypatch.setenv('WAREHOUSE_LOW_STOCK', raw)
    with pytest.raises(ValueError):
        low_stock_threshold()

def test_bad_quantity(csv_factory):
    path = csv_factory('code,name,quantity,unit_price\nA1,Test,wrong,2\n')
    with pytest.raises(ImportRecordError) as exc:
        list(read_items(path))
    assert exc.value.field == 'quantity'
    assert isinstance(exc.value.__cause__, ValueError)

def test_bad_header(csv_factory):
    with pytest.raises(ImportRecordError, match='header'):
        list(read_items(csv_factory('code,name\nA1,Test\n')))

def test_export(tmp_path, item):
    path = tmp_path / 'result.json'
    export_json([item], path)
    assert json.loads(path.read_text(encoding='utf-8'))[0]['value'] ==
pytest.approx(1255)
```

9. **У файлі tests/integration/test_flows.py** створено інтеграційні тести, що перевіряють повний потік роботи системи (імпорт файлу -> оновлення складу -> перевірка запасів -> експорт у JSON) з використанням фікстури tmp_path та monkeypatch.

```
import json
import pytest
from warehouse.io import read_items, export_json
from warehouse.services import Warehouse
from warehouse.config import low_stock_threshold

@pytest.mark.integration
def test_import_to_stock_to_export(csv_factory, tmp_path):
    path =
csv_factory('code,name,quantity,unit_price\nA1,Keyboard,3,100\nB2,Mouse,1,50\n')
    warehouse = Warehouse()
    for item in read_items(path):
        warehouse.add(item)
    warehouse.receive('A1', 2)
    warehouse.issue('B2', 1)
    output = tmp_path / 'warehouse.json'
    export_json(warehouse.items.values(), output)
    data = json.loads(output.read text(encoding='utf-8'))
    assert len(data) == 2
    assert sum(x['value'] for x in data) == pytest.approx(500)
```

```
@pytest.mark.integration
def test_config_and_low_stock(csv_factory, monkeypatch):
    monkeypatch.setenv('WAREHOUSE_LOW_STOCK', '3')
    warehouse = Warehouse()
    for item in
read_items(csv_factory('code,name,quantity,unit_price\nA1,Keyboard,3,100\nB2,Mou
se,4,50\n')):
        warehouse.add(item)
    assert [x.code for x in warehouse.low_stock(low_stock_threshold())] ==
['A1']

@pytest.mark.integration
def test_import_error_preserves_existing_stock(csv_factory, item):
    warehouse = Warehouse()
    warehouse.add(item)
    from warehouse.models import ImportRecordError
    with pytest.raises(ImportRecordError):
        for record in
read_items(csv_factory('code,name,quantity,unit_price\nX,Invalid,not_int,5\n')):
            warehouse.add(record)
    assert warehouse.total_value() == pytest.approx(1255)
```

Результати:

Встановлення залежностей проєкту:

(.venv) PS C:\Users\Анастасія\PycharmProjects\lab6свелеба> python -m pip install -e ".[dev]"

Obtaining file:///C:/Users/%D0%90%D0%BD%D0%B0%D1%81%D1%82%D0%B0%D1%81%D1%96%D1%8F/PycharmProjects/lab6%D1%81%D0%B2%D0%B5%D0%BB%D0%B5%D0%B1%D0%B0

  Installing build dependencies ... done

  Checking if build backend supports build_editable ... done

  Getting requirements to build editable ... done

  Preparing editable metadata (pyproject.toml) ... done

Collecting pytest>=8 (from warehouse-lab06-variant10==0.1.0)

  Using cached pytest-9.1.1-py3-none-any.whl.metadata (7.6 kB)

Collecting pytest-cov>=5 (from warehouse-lab06-variant10==0.1.0)

  Using cached pytest_cov-7.1.0-py3-none-any.whl.metadata (32 kB)

Collecting colorama>=0.4 (from pytest>=8->warehouse-lab06-variant10==0.1.0)

  Using cached colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)

Collecting iniconfig>=1.0.1 (from pytest>=8->warehouse-lab06-variant10==0.1.0)

  Using cached iniconfig-2.3.1-py3-none-any.whl.metadata (2.8 kB)

Collecting packaging>=22 (from pytest>=8->warehouse-lab06-variant10==0.1.0)

  Using cached packaging-26.3-py3-none-any.whl.metadata (3.5 kB)

Collecting pluggy<2,>=1.5 (from pytest>=8->warehouse-lab06-variant10==0.1.0)

  Using cached pluggy-1.7.0-py3-none-any.whl.metadata (5.0 kB)

Collecting pygments>=2.7.2 (from pytest>=8->warehouse-lab06-variant10==0.1.0)

  Using cached pygments-2.21.0-py3-none-any.whl.metadata (2.5 kB)

Collecting coverage>=7.10.6 (from coverage[toml]>=7.10.6->pytest-cov>=5->warehouse-lab06-variant10==0.1.0)

Using cached coverage-7.16.2-cp313-cp313-win_amd64.whl.metadata (8.4 kB)

Using cached pytest-9.1.1-py3-none-any.whl (386 kB)

Using cached pluggy-1.7.0-py3-none-any.whl (27 kB)

Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)

Using cached iniconfig-2.3.1-py3-none-any.whl (7.6 kB)

Using cached packaging-26.3-py3-none-any.whl (129 kB)

Using cached pygments-2.21.0-py3-none-any.whl (1.3 MB)

Using cached pytest_cov-7.1.0-py3-none-any.whl (22 kB)

Using cached coverage-7.16.2-cp313-cp313-win_amd64.whl (226 kB)

Building wheels for collected packages: warehouse-lab06-variant10

Building editable for warehouse-lab06-variant10 (pyproject.toml) ... done

Created wheel for warehouse-lab06-variant10: filename=warehouse_lab06_variant10-0.1.0-0.editable-py3-none-any.whl size=1476 sha256=f93e1fdbcffab5676285954b38ee13e6486a9809cc7a28849277ec6111bebd77

Stored in directory: C:\Users\Анастасія\AppData\Local\Temp\pip-ephem-wheel-cache-p3buwc7y\wheels\0c\d7\07\4b4573da5517cd7af306a79256d4f2bfd67d2383c1deb87895

Successfully built warehouse-lab06-variant10

Installing collected packages: warehouse-lab06-variant10, pygments, pluggy, packaging, iniconfig, coverage, colorama, pytest, pytest-cov

Successfully installed colorama-0.4.6 coverage-7.16.2 iniconfig-2.3.1 packaging-26.3 pluggy-1.7.0 pygments-2.21.0 pytest-9.1.1 pytest-cov-7.1.0 warehouse-lab06-variant10-0.1.0

[notice] A new release of pip is available: 26.1.2 -> 26.2.2

[notice] To update, run: python.exe -m pip install --upgrade pip

Запуск тестів та генерація HTML-звіту покриття:

(.venv) PS C:\Users\Анастасія\PycharmProjects\lab6свелеба> python -m pytest -v --cov=warehouse --cov-branch --cov-report=term-missing --cov-report=html

============================= test session starts ==============================

platform win32 -- Python 3.13.14, pytest-9.1.1, pluggy-1.7.0 -- C:\Users\Анастасія\PycharmProjects\lab6свелеба\.venv\Scripts\python.exe

cachedir: .pytest_cache

rootdir: C:\Users\Анастасія\PycharmProjects\lab6свелеба

configfile: pyproject.toml

testpaths: tests

plugins: cov-7.1.0

collected 48 items

tests/integration/test_flows.py::test_import_to_stock_to_export PASSED [2%]

```
tests/integration/test_flows.py::test_config_and_low_stock PASSED          [  4%]
tests/integration/test_flows.py::test_import_error_preserves_existing_stock PASSED [  6%]
tests/unit/test_io_config.py::test_config_default PASSED                   [  8%]
tests/unit/test_io_config.py::test_config_override PASSED                  [ 10%]
tests/unit/test_io_config.py::test_config_invalid[abc] PASSED              [ 12%]
tests/unit/test_io_config.py::test_config_invalid[-1] PASSED               [ 14%]
tests/unit/test_io_config.py::test_bad_quantity PASSED                     [ 16%]
tests/unit/test_io_config.py::test_bad_header PASSED                       [ 18%]
tests/unit/test_io_config.py::test_export PASSED                           [ 20%]
tests/unit/test_models.py::test_creation PASSED                            [ 22%]
tests/unit/test_models.py::test_value PASSED                               [ 25%]
tests/unit/test_models.py::test_valid_quantity[0] PASSED                   [ 27%]
tests/unit/test_models.py::test_valid_quantity[1] PASSED                   [ 29%]
tests/unit/test_models.py::test_valid_quantity[100] PASSED                 [ 31%]
tests/unit/test_models.py::test_invalid_quantity[-1] PASSED                [ 33%]
tests/unit/test_models.py::test_invalid_quantity[1.2] PASSED               [ 35%]
tests/unit/test_models.py::test_invalid_quantity[True] PASSED              [ 37%]
tests/unit/test_models.py::test_invalid_quantity[2] PASSED                 [ 39%]
tests/unit/test_models.py::test_valid_price[0] PASSED                      [ 41%]
tests/unit/test_models.py::test_valid_price[0.01] PASSED                   [ 43%]
tests/unit/test_models.py::test_valid_price[100.25] PASSED                 [ 45%]
tests/unit/test_models.py::test_invalid_price[-1] PASSED                   [ 47%]
tests/unit/test_models.py::test_invalid_price[inf] PASSED                  [ 50%]
tests/unit/test_models.py::test_invalid_price[nan] PASSED                  [ 52%]
tests/unit/test_models.py::test_invalid_price[True] PASSED                 [ 54%]
tests/unit/test_models.py::test_invalid_price[10] PASSED                   [ 56%]
tests/unit/test_models.py::test_required_fields[-ok] PASSED                [ 58%]
tests/unit/test_models.py::test_required_fields[A-] PASSED                 [ 60%]
tests/unit/test_models.py::test_required_fields[None-ok] PASSED            [ 62%]
tests/unit/test_services.py::test_empty_warehouse PASSED                   [ 64%]
tests/unit/test_services.py::test_find PASSED                              [ 66%]
tests/unit/test_services.py::test_find_missing PASSED                      [ 68%]
tests/unit/test_services.py::test_receive PASSED                           [ 70%]
tests/unit/test_services.py::test_issue PASSED                             [ 72%]
tests/unit/test_services.py::test_insufficient_stock PASSED                [ 75%]
tests/unit/test_services.py::test_invalid_amount[0] PASSED                 [ 77%]
tests/unit/test_services.py::test_invalid_amount[-1] PASSED                [ 79%]
```

tests/unit/test_services.py::test_invalid_amount[2.5] PASSED [81%]

tests/unit/test_services.py::test_invalid_amount[True] PASSED [83%]

tests/unit/test_services.py::test_missing_code PASSED [85%]

tests/unit/test_services.py::test_low_stock PASSED [87%]

tests/unit/test_services.py::test_invalid_threshold PASSED [89%]

tests/unit/test_services.py::test_duplicate PASSED [91%]

tests/unit/test_services.py::test_repository_interaction PASSED [93%]

tests/unit/test_services.py::test_supplier_return_value PASSED [95%]

tests/unit/test_services.py::test_supplier_side_effect PASSED [97%]

tests/unit/test_services.py::test_async_reorder PASSED [100%]

================================ tests coverage ================================

_____________ coverage: platform win32, python 3.13.14-final-0 _____________

Name Stmts Miss Branch BrPart Cover Missing

---

src\warehouse__init__.py 0 0 0 0 100%

src\warehouse\config.py 10 0 2 0 100%

src\warehouse\io.py 25 4 4 0 86% 19-20, 23-24

src\warehouse\models.py 26 0 8 0 100%

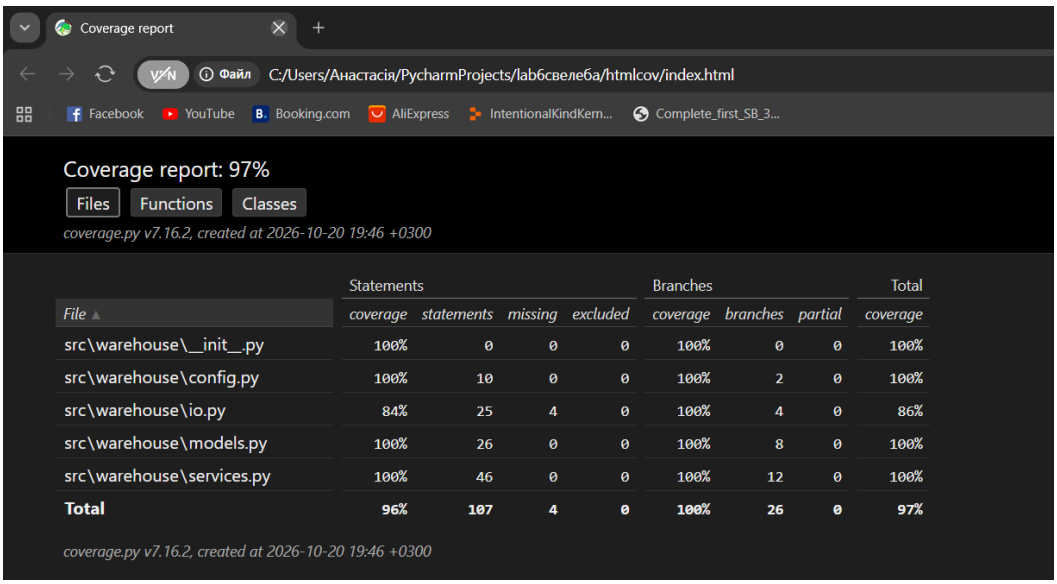
src\warehouse\services.py 46 0 12 0 100%

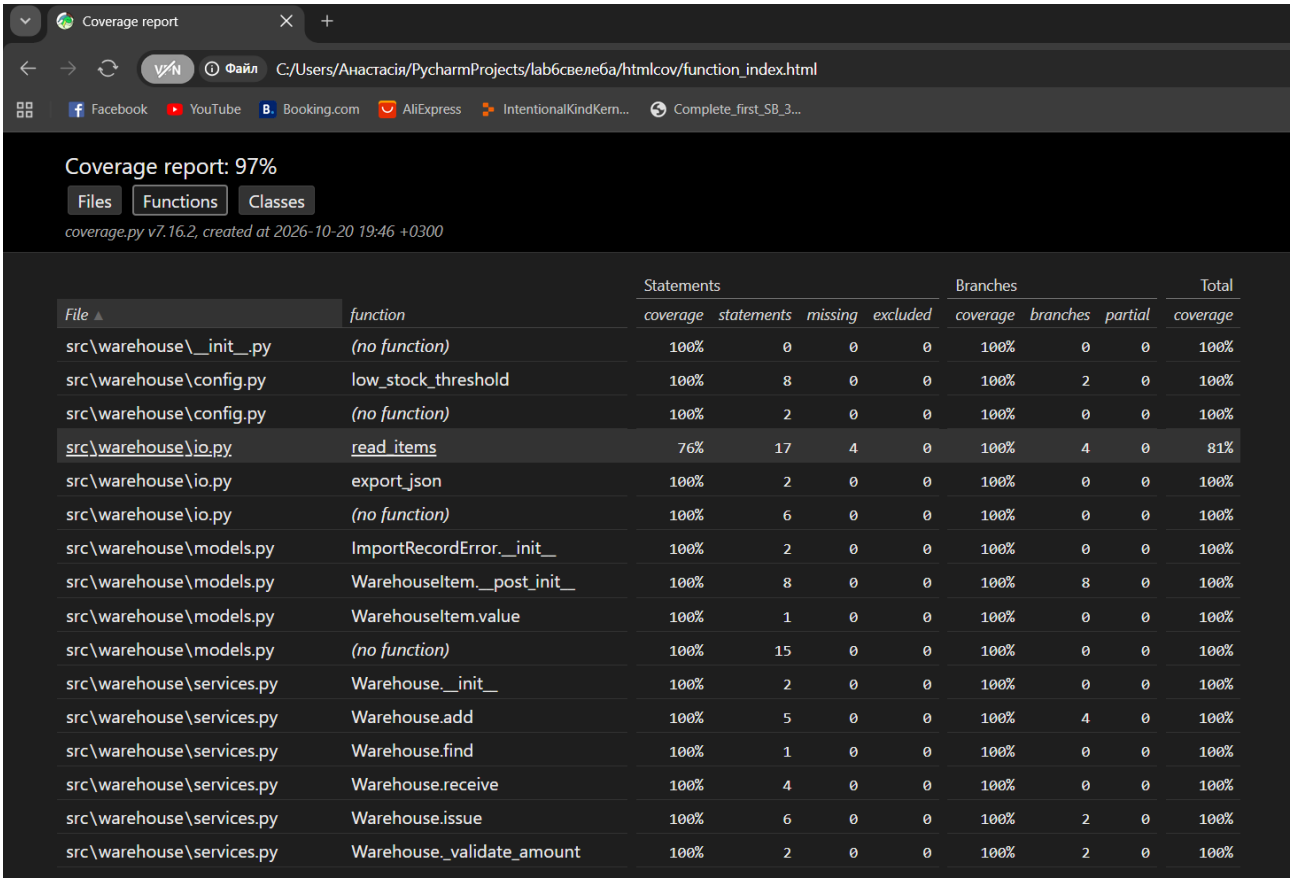
TOTAL 107 4 26 0 97%

Coverage HTML written to dir htmlcov

Required test coverage of 80.0% reached. Total coverage: 96.99%

============================== 48 passed in 0.44s ==============================





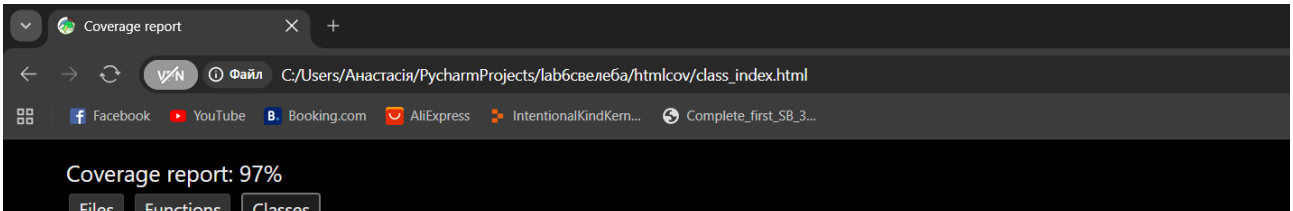
Coverage report: 97%

Files | Functions | Classes

coverage.py v7.16.2, created at 2026-10-20 19:46 +0300

		Statements				Branches			Total
File ▲	function	coverage	statements	missing	excluded	coverage	branches	partial	coverage
src\warehouse__init__.py	*(no function)*	100%	0	0	0	100%	0	0	100%
src\warehouse\config.py	low_stock_threshold	100%	8	0	0	100%	2	0	100%
src\warehouse\config.py	*(no function)*	100%	2	0	0	100%	0	0	100%
src\warehouse\io.py	read_items	76%	17	4	0	100%	4	0	81%
src\warehouse\io.py	export_json	100%	2	0	0	100%	0	0	100%
src\warehouse\io.py	*(no function)*	100%	6	0	0	100%	0	0	100%
src\warehouse\models.py	ImportRecordError.__init__	100%	2	0	0	100%	0	0	100%
src\warehouse\models.py	WarehouseItem.__post_init__	100%	8	0	0	100%	8	0	100%
src\warehouse\models.py	WarehouseItem.value	100%	1	0	0	100%	0	0	100%
src\warehouse\models.py	*(no function)*	100%	15	0	0	100%	0	0	100%
src\warehouse\services.py	Warehouse.__init__	100%	2	0	0	100%	0	0	100%
src\warehouse\services.py	Warehouse.add	100%	5	0	0	100%	4	0	100%
src\warehouse\services.py	Warehouse.find	100%	1	0	0	100%	0	0	100%
src\warehouse\services.py	Warehouse.receive	100%	4	0	0	100%	0	0	100%
src\warehouse\services.py	Warehouse.issue	100%	6	0	0	100%	2	0	100%
src\warehouse\services.py	Warehouse._validate_amount	100%	2	0	0	100%	2	0	100%
src\warehouse\services.py	Warehouse._required	100%	4	0	0	100%	2	0	100%
src\warehouse\services.py	Warehouse.total_value	100%	1	0	0	100%	0	0	100%
src\warehouse\services.py	Warehouse.low_stock	100%	3	0	0	100%	2	0	100%
src\warehouse\services.py	Warehouse.request_reorder	100%	2	0	0	100%	0	0	100%
src\warehouse\services.py	Warehouse.async_reorder	100%	2	0	0	100%	0	0	100%
src\warehouse\services.py	*(no function)*	100%	14	0	0	100%	0	0	100%
Total		**96%**	**107**	**4**	**0**	**100%**	**26**	**0**	**97%**

coverage.py v7.16.2, created at 2026-10-20 19:46 +0300



Coverage report: 97%

Files | Functions | Classes

coverage.py v7.16.2, created at 2026-10-20 19:46 +0300

		Statements				Branches			Total
File ▲	class	coverage	statements	missing	excluded	coverage	branches	partial	coverage
src\warehouse__init__.py	*(no class)*	100%	0	0	0	100%	0	0	100%
src\warehouse\config.py	*(no class)*	100%	10	0	0	100%	2	0	100%
src\warehouse\io.py	*(no class)*	84%	25	4	0	100%	4	0	86%
src\warehouse\models.py	WarehouseError	100%	0	0	0	100%	0	0	100%
src\warehouse\models.py	InsufficientStockError	100%	0	0	0	100%	0	0	100%
src\warehouse\models.py	ImportRecordError	100%	2	0	0	100%	0	0	100%
src\warehouse\models.py	WarehouseItem	100%	9	0	0	100%	8	0	100%
src\warehouse\models.py	*(no class)*	100%	15	0	0	100%	0	0	100%
src\warehouse\services.py	Warehouse	100%	32	0	0	100%	12	0	100%
src\warehouse\services.py	*(no class)*	100%	14	0	0	100%	0	0	100%
Total		**96%**	**107**	**4**	**0**	**100%**	**26**	**0**	**97%**

coverage.py v7.16.2, created at 2026-10-20 19:46 +0300

Висновок: у ході виконання лабораторної роботи я розробила систему керування складськими запасами мовою Python, яка забезпечує реєстрацію товарів, облік їхньої кількості, надходження та списання продукції, обчислення загальної вартості запасів і визначення товарів із низьким залишком. Також я реалізувала імпорт даних із CSV-файлів, експорт інформації у формат JSON, налаштування параметрів системи через змінні середовища та асинхронну взаємодію з постачальником. Для перевірки правильності роботи програми було створено модульні й інтеграційні тести з використанням pytest, фікстур, mock-об’єктів і засобів тестування асинхронного коду. У процесі тестування перевірено коректність роботи основних компонентів системи, обробку некоректних даних і виникнення помилкових ситуацій. У результаті виконання роботи я отримала практичні навички організації Python-проєкту, написання автоматизованих тестів, перевірки взаємодії між компонентами та налаштування контролю покриття коду тестами.